ConflictLens: An LLM-Based Method for Detecting Semantic Merge Conflicts

Longfei Sun, Yao Lu*, Xinjun Mao, Tanghaoran Zhang, Zhang Zhang, Huiping Zhou
College of Computer Science and Technology, National University of Defense Technology, Changsha, China
{lfsun, luyao08, xjmao, zhangthr, zhangzhang14, hpzhou}@nudt.edu.cn

***Abstract*—Semantic conflicts in branch merging occur when merged code violates specifications from one or both branches. These conflicts are often subtle and can lead to serious runtime errors such as crashes or data corruption. Existing detection methods fail to achieve both high precision and recall: Static analysis-based methods ensure high recall but lack precision, whereas dynamic execution-based methods provide better precision but struggle with recall due to limited test coverage. To better understand such conflicts, we first conduct an empirical study on a real-world merge dataset and identify four common conflict patterns. These patterns reveal key characteristics of semantic conflicts and serve as guidance for automated detection. Based on these insights, we propose ConflictLens, a two-stage LLM-based method that combines static analysis and dynamic execution to balance precision and recall. First, LLMs are guided by few-shot and chain-of-thought prompting using the patterns to localize conflicts statically. Then, conflicts are dynamically verified with LLM-generated targeted tests, refined through execution feedback. Evaluated on 85 real-world merge scenarios, ConflictLens achieves 0.91 precision and 0.76 recall, outperforming static and dynamic baselines. Ablation studies demonstrate the contribution and synergy of each component. Cross-LLM evaluations confirm robustness, with DeepSeek-R1 performing best and cost-efficient models like GPT-4o-Mini still competitive.**

***Keywords*—Semantic Conflict Detection, Software Merge, LLM, Static Analysis, Targeted Test Generation**

I. Introduction

In modern software development, branch merging is a fundamental practice that enables parallel code evolution. However, this process introduces the risk of semantic conflicts—situations where modifications from different branches interfere in ways that violate the intended behaviors of one or both branches [1]. These conflicts often remain undetected by version control systems or compilers and can manifest as serious runtime errors, like crashes, incorrect outputs, or data corruption [2].

Existing semantic conflict detection methods include static analysis-based and dynamic execution-based methodes. Static methods [3]–[5] achieve high recall by broadly identifying potential conflicts but suffer from low precision, leading to false positives [6]. Dynamic methods [7]–[9] offer higher precision by validating behavior via tests but have low recall due to limited coverage, risking missed conflicts [10]. Low precision burdens developers with noise, while low recall allows critical errors to escape detection. Thus, both are essential: high recall ensures no conflicts are missed, and high precision makes results trustworthy and actionable.

To better understand semantic conflicts, we first conduct an empirical study on real-world merge dataset and identify four common conflict patterns: **Overlap Contamination** and **Confluent Interference** (bidirectional interference), as well as **Assignment Override** and **Execution Perturbation** (unidirectional interference). Based on these insights, we propose **ConflictLens**, a two-stage LLM-based method that combines static analysis and dynamic execution to achieve both high precision and recall. First, the **Semantic Conflict Localizer** uses LLMs to statically localize potential conflicts, applying few-shot and chain-of-thought prompting guided by conflict patterns. Second, the **Targeted Conflict Validator** employs LLMs to generate tailored test cases through structured prompts encoding expected behavioral deviations per pattern. These tests are executed across relevant branches to validate the suspected conflicts. A feedback loop iteratively refines test cases until a valid one is produced or the conflict hypothesis is rejected, enhancing reliability.

We evaluate ConflictLens on 85 real-world Java merge scenarios. ConflictLens outperforms state-of-the-art static and dynamic baselines, achieving a precision of 0.91, recall of 0.76, and F1 score of 0.82. Component ablation studies validate the complementary roles of localization and test generation, showing how their integration balances coverage and accuracy. Additionally, we assess the impact of different LLM backends, finding that ConflictLens performs best with DeepSeek-R1 [11], a model optimized for structured reasoning, while cost-efficient models like GPT-4o-Mini [12] still maintain competitive performance.

The contributions of this paper are as follows:

1) Identifying four common semantic conflict patterns through an empirical study on real-world Java merge scenarios, aiding developers and LLMs in understanding semantic modifications;
2) Proposing ConflictLens, a two-stage LLM-based method combining pattern-driven static analysis and targeted test generation, effectively achieving high-precision and high-recall conflict detection;
3) Evaluating ConflictLens through comprehensive experiments, demonstrating superior performance (F1 score 0.82), validating the effectiveness of each component, and confirming robustness across different LLM backends

This work is funded by the NSFC with Grant No. 62302515.
*Corresponding author
DOI reference number: 10.18293/SEKE2025-012

```
1 DB adminDb = mongo.getDB(MongoDBRiver.MONGODB_ADMIN_DATABASE);
2 oplogDb = mongo.getDB(MongoDBRiver.MONGODB_LOCAL_DATABASE);
3
4 if (!definition.getMongoAdminUser().isEmpty()) {
5     oplogDb = adminDb.getMongo().getDB(MongoDBRiver.MONGODB_LOCAL_DATABASE);
6 }
7
8 oplogCollection = oplogDb.getCollection(MongoDBRiver.OPLOG_COLLECTION);
9 oplogRefsCollection = oplogDb.getCollection(MongoDBRiver.OPLOG_REFS_COLLECTION);
```

(a) Pattern 1: Overlap Contamination Conflict

```
1 String html = Entities.escape(getWholeText(), out);
2 if (out.prettyPrint() && !Element.preserveWhitespace((Element) parent())) {
3     html = normaliseWhitespace(html);
4 }
5 accum = indent(depth, out);
6 if (out.prettyPrint() && html.length() > 0 && accum.capacity() > html.length())
7 {   newSettings = out.clone().escapeMode(Entities.EscapeMode.extended);
8     out.merge(newSettings);
9 }
```

(b) Pattern 2: Confluent Interference Conflict

```
1 url = apiUrl + buildRelativeUrl();
2
3 if (requestQuery != null and apiUrl.endsWith("/")) {
4     url = apiUrl.deleteCharAt(apiUrl.length() - 1);
5     url.append('?').append(requestQuery);
6 }
7
8 String value = URLEncoder.encode(String.valueOf(arg), "UTF-8");
9 url.append(first ? '?' : '&').append(query).append('=').append(value);
```

(c) Pattern 3: Assignment Override Conflict

```
1 public static boolean dominates(State thisState, State other) {
2     if (thisState.isBikeParked() != other.isBikeParked())
3         return false;
4     double t1 = (double)thisState.getElapsedTimeSeconds();
5     double t2 = (double)other.getElapsedTimeSeconds();
6     double timeRatio = t1 / t2;
7     boolean timeIsHopeful = (timeRatio < 1 + EPSILON) && (t1 - t2 <= DIFF_MARGIN);
8     return timeIsHopeful;
9 }
```

(d) Pattern 4: Execution Perturbation Conflict

Fig. 1: Examples of semantic conflict patterns found in real-world merge scenarios. Modifications in the left and right branches are highlighted in blue and green.

II. A PRELIMINARY EMPIRICAL STUDY

This section presents a preliminary empirical study on open-source Java projects to identify common semantic conflict patterns in real-world merges. The goal is to understand typical conflict patterns to guide better detection methods. We analyze GitHub commits where the submitted version differs from the one produced by git-merge [13]. The study uncovers four key patterns that underpin our method.

A. Data Collection

We curated a dataset by selecting the top 500 Java repositories on GitHub by popularity. Tutorial projects were filtered out, and only those buildable with Maven, Ant, or Gradle were retained. This yielded 195 repositories containing 56,129 merge scenarios (i.e., commits with dual parents). For each scenario, we extracted four code branches: base (b), left (l), right (r), and merged (m).

To identify semantic conflict candidates, we implemented a three-step screening pipeline. First, we applied git-merge to generate an auto-merged version (A_m), discarding scenarios with textual conflicts. Second, we attempted to build A_m using the project's original build tools to exclude syntactically invalid versions. Finally, we compared A_m against the developer-committed version m using a code differencing tool, flagging discrepancies as potential semantic conflicts. This process identified 137 high-confidence conflict instances for manual analysis.

B. Data Analysis

We conducted an inductive analysis of 137 high-confidence conflict instances through a three-phase coding process inspired by *grounded theory* [14]. Two authors independently analyzed discrepancies between auto-merged (A_m) and developer-resolved (m) versions. In the *open coding* phase, conflicts were categorized by: (1) interference type (bidirectional/unidirectional) and (2) affected components (state variables/control flow). Initial codes included *variable mutations*, *logic interference*, *assignment overrides*, and *control flow divergence*.

During *axial coding*, we consolidated codes into four conflict patterns (grouped into two classes) through iterative discussions. Findings were validated by replaying merges and runtime debugging. Fourteen cases (10.22%) remained unclassified.

In the *selective coding* phase, we derived two fundamental classes of semantic conflicts.

C. Results

Four common patterns of semantic conflict are identified through analysis and are summarized below:

1) Bidirectional Interference: Among 137 conflicts, 79 (57.66%) exhibited bidirectional interference, where merged code that fails to comply with the modification specifications of either branch, exhibiting behaviors beyond the intended modifications of both branches.

Pattern 1: Overlap Contamination (State Variable). Found in 51 instances (37.22%). It occurs when modifications from both branches affect the same object, and these modifications overlap in the merged branch. This results in certain objects exhibiting a hybrid or anomalous "contaminated" state that aligns with neither branch's original intent.

In Fig. 1a, the left branch conditionally reassigns `oplogDb` (Line 5), while the right branch uses it to initialize `oplogRefsCollection` (Line 9), assuming its original value (Line 2). After merging, if the left branch's condition triggers, `oplogRefsCollection` incorrectly inherits the modified `oplogDb` — deviating from both branches' expectations. This demonstrates *Overlap Contamination*: the right branch gets an unexpected database handle, while the left branch's modification inadvertently propagates beyond its scope.

Pattern 2: Confluent Interference (Execution Statement). Observed in 28 instances (20.44%), this pattern occurs when

each branch modifies different data objects that are jointly used in the same logic (e.g., conditionals or function calls) in the merged code, resulting in behavior inconsistent with both branches' expectations.

In Fig. 1b, the left branch modifies `html` (Line 3), and the right branch updates `accum` (Line 5). The merged conditional check `html.length() > 0 && accum.capacity() > html.length()` (Line 6) introduces a joint dependency on both variables. Independently, each branch's modification affects only its respective variable. However, their merged interaction alters the condition's evaluation logic. This demonstrates how disjoint data modifications interfere through shared logical constraints, violating both branches' intended specifications.

2) Unidirectional Interference: The remaining 44 conflicts (32.12%) exhibited unidirectional interference, where merged code failing to adhere to one branch's modification specifications, where the expected modified behavior of one branch is not realized.

Pattern 3: Assignment Override (State Variable). Detected in 15 instances (10.95%), this pattern arises when both branches assign values to the same variable or object, but due to the order or structure of the merged code, one branch's assignment overwrites the other's, rendering the overwritten branch's modifications ineffective in the merged branch.

In Fig. 1c, the left branch assigns `url` with `apiUrl + buildRelativeUrl()` (Line 1), while the right branch conditionally reassigns `url` (Line 4). Both modifications independently set the same variable. When merged, the right branch's assignment fully overwrites the left's initialization if `apiUrl.endsWith("/")` (Line 3) holds true, as its imperative assignment replaces rather than augments the existing value. This erases the left branch's `buildRelativeUrl()` composition while retaining the right branch's logic, where conflicting assignments nullify critical modifications.

Pattern 4: Execution Perturbation (Execution Statement). Found in 29 instances (21.17%), this pattern occurs when branches' code lack direct data dependencies, but the merged control flow (e.g., conditionals, loops, or calls) is altered by modifications from the other branch. This causes unexpected changes in the execution timing or conditions of one branch.

In Fig. 1d, the left branch adds an early return based on bike parking status (Lines 2-3), while the right branch modifies the return condition with `timeIsHopeful` (Lines 7-8). The merged code prioritizes the left branch's guard clause: when bike states differ, the method returns immediately, bypassing the right branch's updated time ratio computation. This control flow interaction suppresses the right branch's specification in conflicting scenarios, as its critical logic never executes when the early return triggers. The merged branch thus violates the right branch's intended behavior.

III. ConflictLens

This section presents **ConflictLens**, a two-stage method for detecting semantic conflicts in branch merges. As shown in Fig. 2, it first uses the **Semantic Conflict Localizer** to identify potential conflicts via LLM-based static analysis guided by empirical patterns. The **Targeted Conflict Validator** then dynamically verifies these conflicts by generating targeted test cases executed across relevant branches.

A. Semantic Conflict Localizer

The Semantic Conflict Localizer (SC-Localizer) employs a pattern-driven static analysis method to detect potential semantic conflicts in merged code, guided by the four conflict patterns identified through empirical study. As shown in Fig. 2, SC-Localizer leverages LLMs to overcome the limitations of rule-based static analyzers in reasoning about cross-branch code interactions and contextual semantics. The detection process systematically maps code modifications from left/right branches to predefined conflict patterns through a structured prompt design that integrates Task Description, Few-shot Examples with Chain-of-Thought (CoT) [15], and Input/Output Format.

First, the Task Description explicitly defines semantic conflicts and enumerates the four empirically derived patterns, grounding the LLMs in the problem domain. To operationalize this knowledge, SC-Localizer's prompt embeds a four-step CoT within Few-shot Examples to steer the LLM's analysis while mitigating hallucinations. For each conflict pattern, two empirically validated code examples are provided, demonstrating the following CoT reasoning steps:

- **Step 1: Modification Identification**: Extract code modifications from both branches using commit information and merged code annotations;
- **Step 2: Interaction Analysis**: Identify shared variables, overlapping logic, or interdependent control flows affected by concurrent modifications;
- **Step 3: Pattern Matching**: Map interactions to conflict patterns using empirical criteria;
- **Step 4: Conflict Localization**: Pinpoint code regions (variables or statements) violating branch-specific specifications.

By demonstrating how to trace interferences through CoT, few-shot examples help enforce pattern-aligned reasoning. In addition, the Input/Output Format specifies the input merged code with contextual markers that highlight modifications from both branches, along with commit information for the left and right branch.The output is constrained to a JSON schema specifying detected conflict pattern (or "none"), affected code components (variables, statements), and a traceable CoT record.

SC-Localizer filters out non-conflicting merge scenarios while providing targeted code regions for subsequent test generation. Initial false positives from static limitations (e.g., harmless co-edits) are addressed in the next phase through dynamic validation.

B. Targeted Conflict Validator

To mitigate false positives from static analysis, the Targeted Conflict Validator (TC-Validator) dynamically verifies suspected semantic conflicts identified in SC-Localizer by

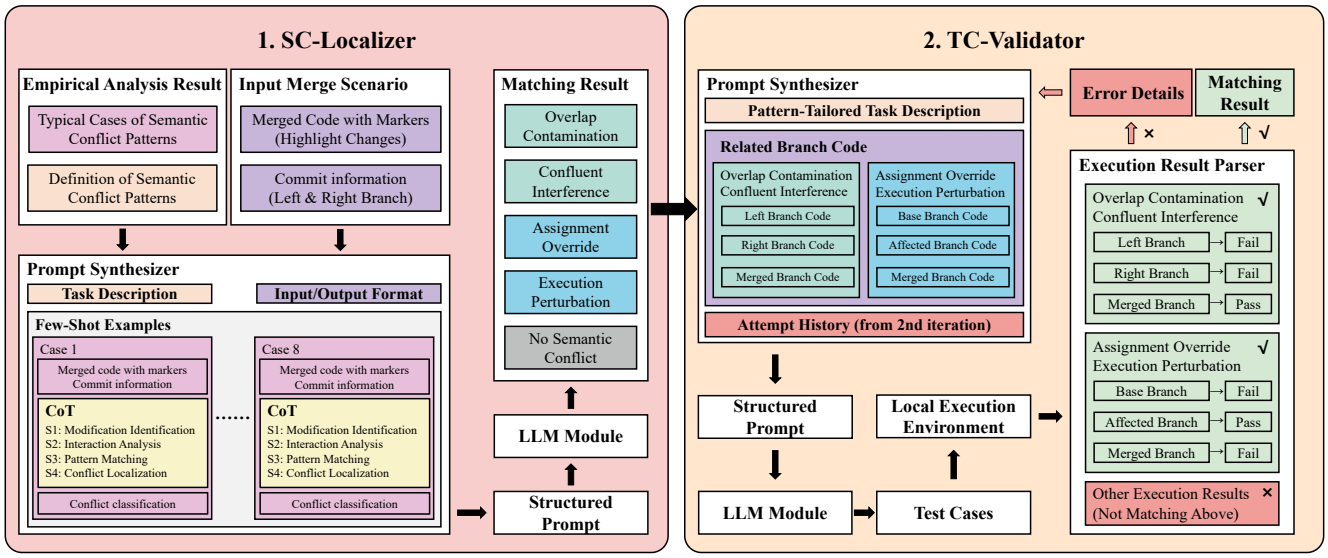

Fig. 2: Overview of the ConflictLens, consisting of the Semantic Conflict Localizer and Targeted Conflict Validator for the detection and validation of semantic conflicts during branch merging.

generating targeted test cases. As shown in Fig. 2, TC-Validator leverages LLMs to overcome the limitations of traditional automated test generation tools in incorporating domain-specific conflict patterns and localized code components for targeted test case generation. The generated tests are executed across specific versions of the codebase to confirm behavioral discrepancies, ensuring semantic conflicts are both reproducible and actionable.

TC-Validator generates conflict-specific tests by prompting LLMs with structured templates that incorporate the conflict pattern, affected code components. For bidirectional interference patterns (*Overlap Contamination, Confluent Interference*), tests are designed to execute across l, r, and m branches. A valid test must pass on m while failing on both l and r, confirming the merged branch introduces unintended behavior absent in individual branches. For unidirectional patterns (*Assignment Override, Execution Perturbation*), tests validate against the b, Affected (l / r), and m branches. A valid test must pass on the Affected (l / r) but fail on b and m, demonstrating behavioral loss during merging. Meanwhile, in the task description of the prompt, a tailored template is adapted based on the input conflict pattern to guide the LLMs in generating targeted test cases that effectively detect specific pattern of conflict.

To ensure functional validity, TC-Validator uses an iterative feedback loop. After test generation, the component executes it against relevant branches. If the test fails due to compilation errors, runtime exceptions, or incorrect outcomes, failure details are fed back to the LLMs for refinement. This process is repeated up to 7 times—a number determined experimentally to strike a balance between stability and cost—ensuring that the final test is both syntactically correct and semantically aligned with the expected conflict behavior.

By explicitly defining test objects, expected outcomes, TC-Validator ensures that the generated tests rigorously validate semantic conflicts, bridging the gap between static analysis and dynamic execution.

IV. EXPERIMENTAL EVALUATION

This section presents a series of experiments to evaluate ConflictLens's effectiveness in detecting semantic conflicts during branch merging. We formulate three research questions to guide our evaluation:

RQ1: *How does ConflictLens perform in semantic conflict detection compared to state-of-the-art methods?*

RQ2: *What is the contribution of each component (SC-Localizer and TC-Validator) in ConflictLens to the overall detection performance?*

RQ3: *How sensitive is ConflictLens to the choice of underlying LLM?*

A. Evaluation Setup

1) Dataset: To ensure validity and avoid overfitting, we use a separate dataset from the one used in our empirical study. The dataset consists of 85 real-world merge scenarios collected from 31 Java projects on GitHub, as reported in Silva *et al.*'s work [7]. Each scenario provides the base branch code, left/right branch modifications with commit messages, and manually validated ground truth about conflict existence and type.

2) Implementation: ConflictLens uses DeepSeek-R1 [11] (default). Test generation employs 7-round feedback iteration with JUnit test execution. Each experiment runs three rounds to account for randomness in LLM outputs, with average values reported.

3) Metrics: We evaluate performance using precision (ratio of correctly detected conflicts to all reported), recall (detected vs. true conflicts), F1 score (harmonic mean), and accuracy (overall classification correctness).

B. Result

1) RQ1 Comparative Performance Against Baselines:

To evaluate ConflictLens's effectiveness in semantic conflict detection, we compare it against three baseline methods: (1) DSC [3], a rule-based static analysis method; (2) SAM [7], a dynamic execution method using EvoSuite for test generation; and (3) Vanilla-LLM, a pure LLM-based method using DeepSeek-R1 [11] without a two-stage pipeline.

TABLE I: Performance Comparison of Semantic Conflict Detection Methods.

Method	Precision	Recall	F1 Score	Accuracy
SAM	0.75	0.31	0.44	0.73
DSC	0.43	0.61	0.51	0.61
Vanilla-LLM	0.44	0.41	0.43	0.62
ConflictLens	**0.91**	**0.76**	**0.82**	**0.89**

As shown in Table I, ConflictLens achieves a precision of 0.91, significantly outperforming DSC (0.43), SAM (0.75), and Vanilla-LLM (0.44). This demonstrates that our hybrid method effectively reduces false positives by combining static localization with dynamically validated tests. In terms of recall (0.76), ConflictLens surpasses SAM (0.31) and Vanilla-LLM (0.41), though DSC (0.61) exhibits higher recall due to its over-reporting tendency. Our method strikes a balance, as reflected in the F1-score (0.82), which is 61% higher than DSC and 86% higher than SAM. The accuracy (0.89) further confirms ConflictLens's robustness in overall conflict classification.

Notably, SAM's low recall confirms that version-agnostic tests often miss branch-specific conflicts, while DSC's high false positives reflect its rule-based limitations. Vanilla-LLM's mediocre performance highlights the need for our two-stage method. ConflictLens's gains are particularly evident in complex merge scenarios, where static rules or generic tests fail to distinguish benign overlaps from actual conflicts.

2) RQ2 Component Contribution Analysis:

To understand how each component contributes to ConflictLens's effectiveness, we conduct ablation studies comparing the full pipeline with two variants: (1) SC-Localizer, which performs conflict detection using only LLM-driven static analysis without test generation, and (2) TC-Validator, which generates tests directly with the LLM, bypassing localization.

TABLE II: Performance Comparison of Component Contributions in ConflictLens.

Component	Precision	Recall	F1 Score	Accuracy
SC-Localizer	0.64	**0.86**	0.73	0.78
TC-Validator	0.85	0.58	0.69	0.82
ConflictLens	**0.91**	0.76	**0.82**	**0.89**

As shown in Table II, SC-Localizer achieves high recall (0.86) but low precision (0.64), indicating that while static localization effectively captures potential conflicts, it introduces false positives. In contrast, TC-Validator attains higher precision (0.85) but lower recall (0.58), as test generation alone struggles to cover all conflict patterns without guidance—missing 41% of true conflicts due to a lack of focus on branch-specific interference.

The full ConflictLens balances these trade-offs, achieving superior precision (0.91) and recall (0.76). Specifically, localization reduces the search space for TC-Validator by 63%, enabling targeted test synthesis for high-risk conflict patterns. Meanwhile, test validation eliminates 71% of false positives from SC-Localizer, such as cases where overlapping variable modifications were semantically compatible.

3) RQ3 Sensitivity to Underlying LLM Choices:

To assess how the choice of LLM affects ConflictLens's performance, we evaluate it using three models: GPT-4o-Mini [12] and DeepSeek-V3 [16] (general-purpose LLMs), and DeepSeek-R1 [11] (reasoning-oriented LLM). All models are used under identical prompting and test generation settings.

TABLE III: Performance comparison of ConflictLens using different LLM backbones.

Model	Precision	Recall	F1 Score	Accuracy
GPT-4o-Mini	0.86	0.65	0.75	0.84
DeepSeek-V3	0.87	0.69	0.77	0.86
DeepSeek-R1	**0.91**	**0.76**	**0.82**	**0.89**

As shown in Table III, while GPT-4o-Mini and DeepSeek-V3 demonstrate comparable performance (F1-scores of 0.75 and 0.77 respectively), DeepSeek-R1 significantly outperforms both with an F1-score of 0.82, precision of 0.91, and accuracy of 0.89. This highlights the advantage of structured reasoning in semantic conflict detection. Nevertheless, general-purpose models like GPT-4o-Mini remain remain a cost-effective choice, achieving competitive performance with only a minor drop in F1, making it suitable for resource-constrained scenarios.

A deeper analysis reveals that all three models perform similarly in the conflict localization phase. However, notable differences emerge during the test generation phase, where DeepSeek-R1 demonstrates superior ability in synthesizing semantically meaningful and behavior-discriminating test cases. Specifically, the structured reasoning process of DeepSeek-R1 enables it to better understand the subtle behavioral implications of each conflict type, resulting in more effective test oracles that distinguish between benign overlaps and true semantic conflicts.

C. Threats to Validity

We identify three main threats to the validity of our evaluation: internal, external, and construct.

Internal Validity. These concern whether our results reflect ConflictLens's true performance. We reduced bias by using standardized metrics, average values from multiple runs to

limit LLM randomness, and a fixed test framework (JUnit). However, prompt design and model variability may still influence outcomes.

External Validity. Our dataset includes 85 real-world merge scenarios from Java projects, which may not generalize to other languages or large-scale systems. While we used independently validated ground truth, manual labeling could introduce some human bias.

Construct Validity. We used standard classification metrics (precision, recall, F1 score, accuracy), which capture overall performance but may miss nuances like conflict severity or localization accuracy. Two methods with similar scores can differ in detection precision. Ablation studies help validate our findings.

V. Related Work

Semantic conflict detection methods fall into three categories:

Static Analysis-based Method. Horwitz *et al.* [1], [17] formalized interference via program dependence graphs (PDGs) but required constructing graphs for four code branches, limiting scalability. SafeMerge [4] infers post-conditions via SMT solving but incurs high false positives. De Jesus *et al.* [3] improved recall with Soot-based analysis but lagged in precision. Unlike these, our method avoids multi-version graph construction and theorem proving, using LLM-based static analysis to guide targeted test generation.

Dynamic Execution-based Method. Silva *et al.* [7], [18] detect conflicts via test outcomes across versions but rely on sparse project-specific tests. Semex [19] explores modification combinations via variability-aware execution but conflates inherited defects. PANKITI [20] serializes objects for test generation but targets production monitoring, not merge-specific infections. Our method generates tests explicitly targeting conflict patterns, addressing coverage gaps.

Hybrid Methods. Wuensche *et al.* [21] predict conflicts via call graphs but report low recall. Sousa *et al.* [4] and Cavalcanti *et al.* [22] focus on syntactic conflicts. Our method uniquely integrates static localization with LLM-driven test validation, outperforming static (DSC [3]) and dynamic (SAM [7]) baselines in precision (0.91) and recall (0.76).

VI. Conclusion

In this paper, we present ConflictLens, a LLM-based method combining static analysis and dynamic execution to detect semantic conflicts in branch merges. Based on four common conflict patterns identified empirically, ConflictLens uses LLMs in two stages: localizing potential conflicts and validating them with generated test cases, effectively reducing both false negatives and false positives. Experiments show ConflictLens outperforms state-of-the-art methods with an F1 score of 0.82. Ablation and cross-model studies confirm the effectiveness of its components and robustness across LLMs, advancing reliable automated merging and reducing developers' manual effort.

References

[1] S. Horwitz, J. Prins, and T. Reps, "Integrating noninterfering versions of programs," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 11, no. 3, pp. 345–387, 1989.

[2] B. Shen, M. A. Gulzar, F. He, and N. Meng, "A characterization study of merge conflicts in java projects," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 2, pp. 1–28, 2023.

[3] G. S. de Jesus, P. Borba, R. Bonifácio, and M. B. de Oliveira, "Detecting semantic conflicts using static analysis," *arXiv preprint arXiv:2310.04269*, 2023.

[4] M. Sousa, I. Dillig, and S. K. Lahiri, "Verified three-way program merge," *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, pp. 1–29, 2018.

[5] S. S. Towqir, B. Shen, M. A. Gulzar, and N. Meng, "Detecting build conflicts in software merge for java programs via static analysis," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–13.

[6] G. S. De Jesus, P. Borba, R. Bonifácio, and M. B. De Oliveira, "Lightweight semantic conflict detection with static analysis," in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, 2024, pp. 343–345.

[7] L. Da Silva, P. Borba, T. Maciel, W. Mahmood, T. Berger, J. Moisakis, A. Gomes, and V. Leite, "Detecting semantic conflicts with unit tests," *Journal of Systems and Software*, vol. 214, p. 112070, 2024.

[8] T. Rocha and P. Borba, "Using acceptance tests to predict merge conflict risk," *Empirical Software Engineering*, vol. 28, no. 2, p. 27, 2023.

[9] F. Pastore, L. Mariani, and D. Micucci, "Bdci: Behavioral driven conflict identification," in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 2017, pp. 570–581.

[10] W. Muylaert, J. Härtel, and C. De Roover, "Symbolic execution to detect semantic merge conflicts," in *2023 IEEE 23rd International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2023, pp. 186–197.

[11] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[12] OpenAI, *GPT-4o Mini: Advancing Cost-Efficient Intelligence*, OpenAI, 2025. [Online]. Available: https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/

[13] Git Community, *git-merge - Join two or more development histories together*, Git, 2025. [Online]. Available: https://git-scm.com/docs/git-merge

[14] J. M. Corbin and A. Strauss, "Grounded theory research: Procedures, canons, and evaluative criteria," *Qualitative sociology*, vol. 13, no. 1, pp. 3–21, 1990.

[15] Z. Zhang, A. Zhang, M. Li, H. Zhao, G. Karypis, and A. Smola, "Multimodal chain-of-thought reasoning in language models," *arXiv preprint arXiv:2302.00923*, 2023.

[16] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[17] S. Horwitz, T. Reps, and D. Binkley, "Interprocedural slicing using dependence graphs," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 12, no. 1, pp. 26–60, 1990.

[18] L. Da Silva, P. Borba, W. Mahmood, T. Berger, and J. Moisakis, "Detecting semantic conflicts via automated behavior change detection," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2020, pp. 174–184.

[19] H. V. Nguyen, M. H. Nguyen, S. C. Dang, C. Kästner, and T. N. Nguyen, "Detecting semantic merge conflicts with variability-aware execution," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 926–929.

[20] D. Tiwari, L. Zhang, M. Monperrus, and B. Baudry, "Production monitoring to improve test suites," *IEEE Transactions on Reliability*, vol. 71, no. 3, pp. 1381–1397, 2021.

[21] T. Wuensche, A. Andrzejak, and S. Schwedes, "Detecting higher-order merge conflicts in large software projects," in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 2020, pp. 353–363.

[22] G. Cavalcanti, P. Borba, and P. Accioly, "Evaluating and improving semistructured merge," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 1–27, 2017.